

React Programming Patterns

Stateful and Stateless Components

In React, a *stateful* component is a component that holds some state. *Stateless* components, by contrast, have no state. Note that both types of components can use props.

In the example, there are two React components. The `Store` component is stateful and the `Week` component is stateless.

```
function Store() {  
  const [sell, setSell] = useState("anything");  
  
  return <h1>I'm selling {sell}</h1>;  
  
}  
  
function Week(props) {  
  
  return <h1>Today is {props.day}</h1>;  
  
}
```

React Programming Pattern

One of the most common programming patterns in React is to use stateful parent components to maintain their own state and pass it down to one or more stateless child components as props. The example code shows a basic example.

```
// This is a stateless child component.  
function BabyYoda(props) {  
  return <h2>I am {props.name}!</h2>;  
}  
  
// This is a stateful Parent element.  
function Yoda() {  
  const [ name, setName ] = useState("Toyoda")  
  
  // The child component will render information passed  
  down from the parent component.  
  return <BabyYoda name={name} />;  
}
```

Changing Props and State

In React, a component should never change its own props directly. A parent component should change them.

State, on the other hand, is the opposite of props: a component keeps track of its own state and can change it at any time.

The example code shows a component that accepts a prop, `subtitle`, which never changes. It also has a state object which does change.

```
function Clock(props) {  
  const [ date, setDate ] = useState(new Date());  
  
  const updateTime = () => {  
    setDate(new Date());  
  }  
  
  return (  
    <div>  
      <h1>It is currently {date.toLocaleTimeString()}  
</h1>  
      <h2>{props.subtitle}</h2>  
      <button onClick={updateTime}>Update the  
clock</button>  
    </div>  
  );  
}
```

Passing State Change Functions as Props

If a React parent component defines a function that changes its state, that function can be passed to a child component and called within the component to updating the parent component's state.

In this example, because `this.setState()` causes the `Name` component to re-render, any change to the `<input>` will update the `Name` component's state, causing a new render and displaying the new state value to the `<p>` tag content.

```
function Name() {  
  const [name, setName] = useState('');  
  
  const handleNameChange = (e) => {  
    setName(e.target.value);  
  }  
  
  return (  
    <div>  
      <input onChange={handleNameChange} />  
      <p>{name}</p>  
    </div>  
  );  
}
```

Event Handlers and State in React

Event handler functions in React are often used to update state. These handler functions often receive an event as an argument, which is used to update state values correctly.

In the example code, we use `event.target.value` to get the input's value.

```
function MyComponent() {  
  const [ text, setText ] = useState("");  
  
  const handleChange(event) => {  
    setText(event.target.value);  
  }  
  
  return (  
    <div>  
      <input onChange={handleChange} value={text} />  
      <p>You typed {text}</p>  
    </div>  
  );  
}
```

Presentational and Container Components

A common programming pattern in React is to have presentational and container components. Container components contain business logic (methods) and handle state. Presentational components render that behavior and state to the user.

In the example code, `CounterContainer` is a container component and `Counter` is a presentational component.

```
class CounterContainer extends React.Component {
  constructor(props) {
    super(props);
    this.state = { count: 0 };
    this.increment = this.increment.bind(this);
  }

  increment() {
    this.setState((oldState) => {
      return { count: oldState.count + 1 };
    });
  }

  render() {
    return <Counter count={this.state.count} increment=
{this.increment} />;
  }
}

class Counter extends React.Component {
  render() {
    return (
      <div>
        <p>The count is {this.props.count}</p>
        <button onClick={this.props.increment}>Add
1</button>
      </div>
    );
  }
}
```